



**FATİH
SULTAN
MEHMET**
VAKIF ÜNİVERSİTESİ

Course: Computer Networks
Prof.Dr.ALİ YILMAZ ÇAMURCU
Arş. Gör. Ertuğrul İSLAMOĞLU

Online Chess

Multiplayer Chess Application Project Report

Group: 27 Hasan Farman (2321021361) and Ahmad Munir Malak (2321021363)

Project Overview

The Online Chess application is a fully functional two-player multiplayer game developed in Python. The project implements the complete FIDE rules of chess, including special mechanics such as castling, en passant, and pawn promotion.

To fulfill the networking requirements of the project, the game was extended into a networked client-server architecture using low-level TCP sockets. Two human players connect to a central relay server, allowing them to play against each other over a local network or the internet in real-time. The graphical user interface (GUI) was developed using the PyQt6 library.

System Architecture & Networking

a)Client-Server Model

The system operates on a strict client-server architecture. The environment consists of two client applications (`main.py`) communicating through a central relay server (`server.py`). The server is designed to be lightweight and stateless; it does not contain any chess game logic.

All game rules, move validations, and state changes are enforced completely on the client side. This architecture minimizes server load and reduces bandwidth usage, as the server's sole responsibility is to pass serialized move data between Player 0 (White) and Player 1 (Black).

b)TCP Communication & Protocol

The network layer utilizes Python's standard `socket` library to establish a persistent TCP connection. Communication between the clients and the server uses a newline-delimited JSON protocol. Every interaction is sent as a JSON object, terminated by a `\n` character, ensuring that messages are parsed cleanly from the TCP stream.

A standard move payload over the network looks like this:

JSON

```
{
  "type": "move",
  "from_x": 4,
  "from_y": 6,
  "to_x": 4,
  "to_y": 4,
  "promotion": null
}
```

c)Threading and Concurrency

To prevent the application's GUI from freezing while waiting for network data, the `NetworkClient` implements a background listener thread. This thread constantly listens for incoming TCP packets and decodes them. When a message (such as an opponent's move) is received, it is dispatched to the main application using PyQt6 signals (`move_received`).

Game Logic & User Interface

a) Graphical Interface

The presentation layer is built using PyQt6 widgets, transitioning smoothly from a Start Screen (for server IP and port configuration) to the active Game Screen, and finally an End Screen. The board is rendered interactively, providing visual feedback such as legal move highlights, capture rings, and a red indicator when a king is in check.



Figure 1: Client-side interface showing the White player's perspective, legal move highlights, and the turn indicator.

b) Board Flipping

To provide an intuitive experience for the second player, the application implements a client-side board flip. When a player is assigned the Black pieces, a boolean flag transforms the coordinate conversion logic, rendering the board rotated 180 degrees without altering the underlying data arrays.



Figure 2: The Black player's perspective, demonstrating the client-side 180-degree board rotation.

Deployment & Testing

To ensure the application functions correctly over a Wide Area Network (WAN), the relay server (`server.py`) was deployed on an Amazon Web Services (AWS) EC2 instance. The server listens on port 5000. Security group rules were configured to allow inbound custom TCP traffic on port 5000 from all IPv4 addresses (`0.0.0.0/0`).

Testing confirmed that latency remained low and that TCP packets were successfully relayed between clients on different local networks without packet loss or UI freezing.

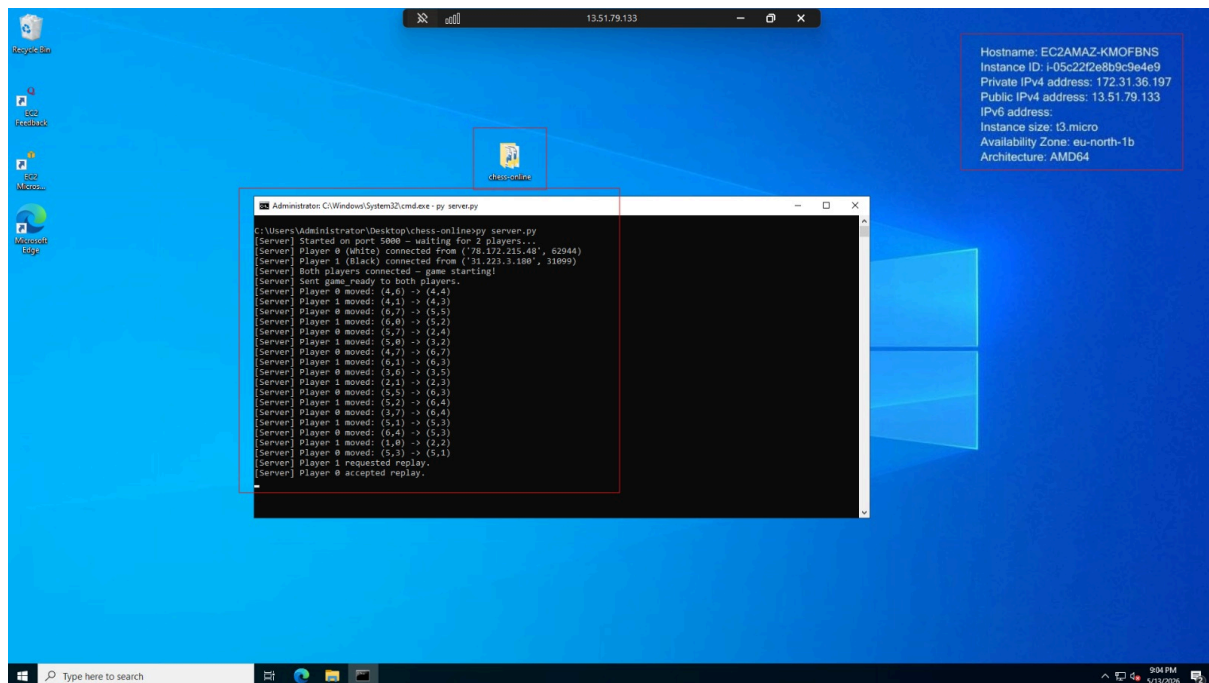


Figure 3: The Python relay server actively running on the AWS EC2 instance, logging connections and move coordinates from both remote clients.

Challenges & Solutions

During development, integrating the network layer with the graphical interface and complex game rules presented several challenges:

Thread-Safe GUI Updates: Initially, calling GUI updates directly from the network listener thread caused application crashes due to undefined behavior.

Solution: We routed all network events through PyQt6 signals. Because Qt automatically queues cross-thread signals, the GUI safely executed the board updates on the main event loop.

Legal-Move Verification Overhead: Validating check/checkmate requires simulating every pseudo-legal move, which is computationally expensive ($O(n^2)$).

Solution: We implemented a lightweight board cloning method using shallow copies (`copy.copy`) of piece objects, simulating candidate moves without bogging down the CPU or requiring a full engine restart.

Conclusion

This project successfully demonstrates the integration of complex application logic with reliable network communication. By separating the GUI, the chess engine, and the TCP networking layer into distinct modules, the codebase remained clean and maintainable. Deploying the relay server on AWS proved that the JSON-over-TCP architecture functions efficiently across the internet, fulfilling all project requirements.